

Index

Sr.no	Practical's	Signature
1.	Create basic FastAPI endpoints	
2.	Handle query and path parameters	
3.	Validate data with Pydantic models	
4.	Create nested input/output models	
5.	Build CRUD APIs for a task management app	
6.	Add custom exception handling	
7.	Add background task to send async email	
8.	Secure routes with user roles	
9.	Write integration test with Pytest/ httpx	
10.	Create async DB APIs with Tortoise ORM	

Practical: 1

Aim -Create basic FastAPI endpoints

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
def home():
```

```
    return {
        "message": "Welcome to FastAPI"
    }
```

```
@app.get("/students")
def get_students():
```

```
    students = [

        {
            "id":1,
            "name":"Nitish",
            "course":"AI"
        },

        {
            "id":2,
            "name":"Rahul",
            "course":"Data Science"
        },

        {
            "id":3,
            "name":"Amit",
            "course":"Cyber Security"
        }
    ]

    return students
```

Output:

FastAPI 0.1.0 OAS 3.1
[/openapi.json](#)

default

GET / Home

GET /students Get Students

GET /students Get Students

Parameters

No parameters

Execute

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

Practical: 2**Aim- Handle Query and Path parameters**

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
students = {  
    1: {"name": "Nitish", "course": "AI", "city": "Mumbai"},  
    2: {"name": "Rahul", "course": "Data Science", "city": "Pune"},  
    3: {"name": "Amit", "course": "Cyber Security", "city": "Delhi"}  
}
```

```
# Home Endpoint
```

```
@app.get("/")  
def home():  
    return {"message": "Welcome to Student API"}
```

```
# Get All Students
```

```
@app.get("/students")  
def get_students():  
    return students
```

```
# Get Student by Path Parameter
```

```
@app.get("/students/{student_id}")  
def get_student(student_id: int):  
    return students.get(student_id, "Student Not Found")
```

```
# Search Students using Query Parameters
```

```
@app.get("/search")  
def search_students(course: str = None, city: str = None):  
    return {  
        "course": course,  
        "city": city  
    }
```

```
# Path + Query Parameter Together
```

```
@app.get("/student/{student_id}/details")  
def student_details(student_id: int, semester: int, section: str):  
    return {  
        "student_id": student_id,  
        "semester": semester,  
        "section": section  
    }
```

FastAPI 0.1.0 OAS 3.1
/openapi.json

default

- GET / Home
- GET /students Get Students
- GET /students/{student_id} Get Student
- GET /search Search Students
- GET /student/{student_id}/details Student Details

student_id * required
integer (path)

1

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/students/1' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/students/1
```

Server response

Code Details

200

Response body

```
{
  "name": "Nitish",
  "course": "AI",
  "email": "nitish@example.com"
}
```

Practical: 3

Aim – Validate date with Pydantic models

```
from fastapi import FastAPI
from pydantic import BaseModel, Field, EmailStr

app = FastAPI()

class Student(BaseModel):

    name: str = Field(
        min_length=3,
        max_length=30
    )

    age: int = Field(
        gt=18,
        lt=60
    )

    email: EmailStr

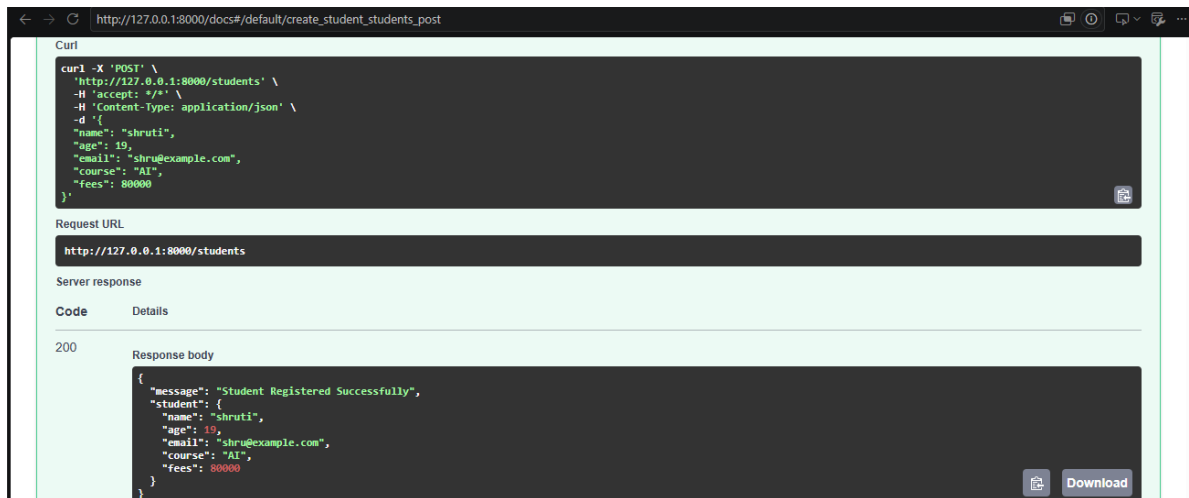
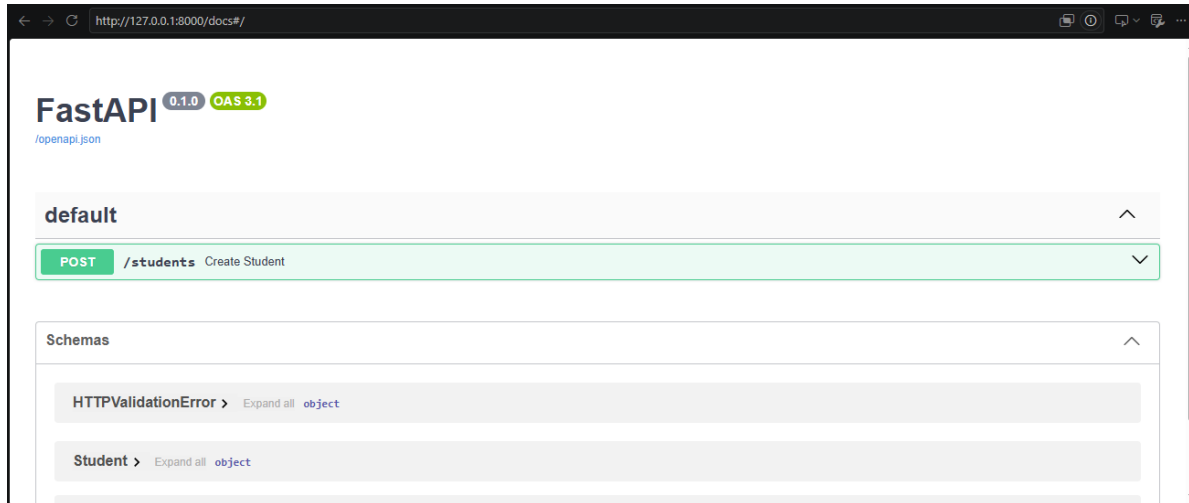
    course: str = Field(
        min_length=2,
        max_length=20
    )

    fees: float

@app.post("/students")
def create_student(student: Student):

    return {
        "message": "Student Registered Successfully",
        "student": student
    }
```

Output:



Practical: 4

Aim – Create Nested input / output models

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# -----
# Address Models
# -----

class AddressInput(BaseModel):
    city: str
    state: str
    pincode: int

class AddressOutput(BaseModel):
    city: str
    state: str

# -----
# Student Models
# -----

class StudentInput(BaseModel):
    name: str
    age: int
    email: str
    address: AddressInput

class StudentOutput(BaseModel):
    name: str
    age: int
    address: AddressOutput

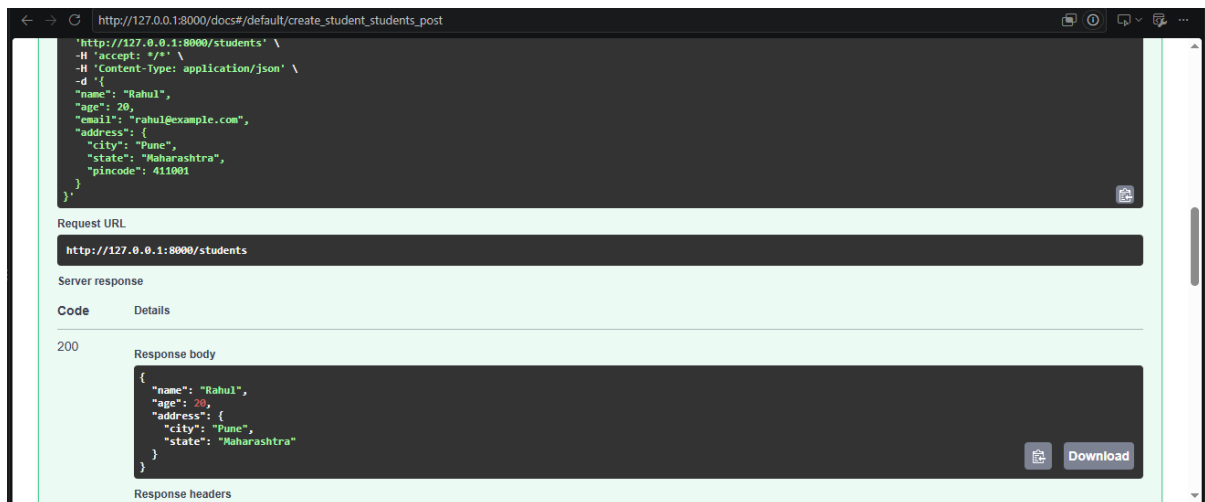
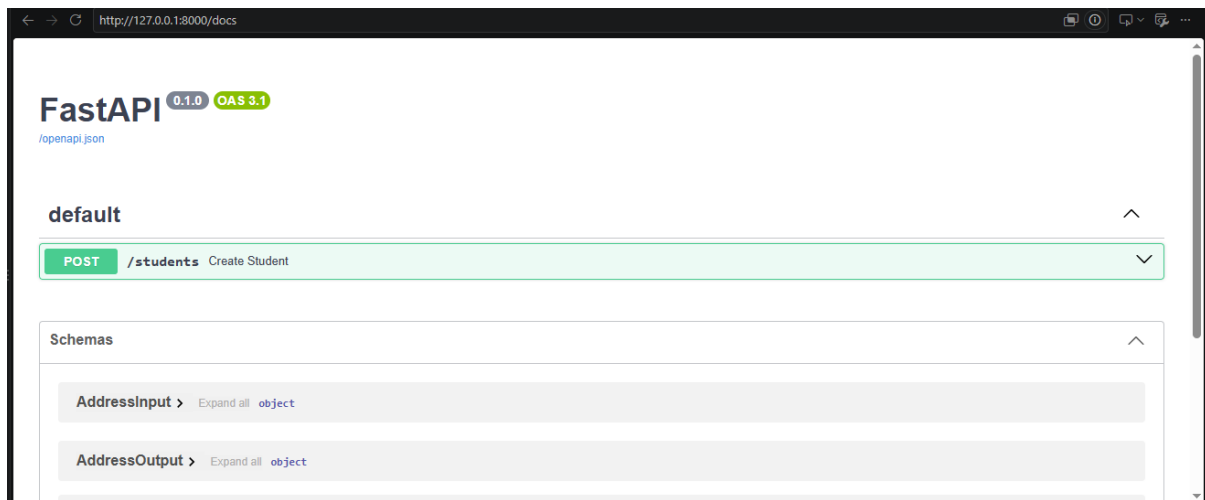
# -----
# API
# -----

@app.post("/students", response_model=StudentOutput)
def create_student(student: StudentInput):

    return {
```

```
"name": student.name,  
"age": student.age,  
"address": {  
    "city": student.address.city,  
    "state": student.address.state  
}  
}
```

Output:



Practical: 5

Aim – Build CRUD APIs for a task management app.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# Task Model
class Task(BaseModel):
    title: str
    completed: bool

# List to store tasks
tasks = []

# Home API
@app.get("/")
def home():
    return {"message": "Task Management API"}

# Create Task
@app.post("/tasks")
def create_task(task: Task):

    tasks.append(task)

    return {
        "message": "Task Added Successfully",
        "task": task
    }

# Get All Tasks
@app.get("/tasks")
def get_tasks():

    return tasks

# Update Task
@app.put("/tasks/{index}")
def update_task(index: int, task: Task):

    tasks[index] = task
```

```
return {  
    "message": "Task Updated",  
    "task": task  
}
```

Delete Task

```
@app.delete("/tasks/{index}")
```

```
def delete_task(index: int):
```

```
    tasks.pop(index)
```

```
    return {  
        "message": "Task Deleted Successfully"  
    }
```

Output:

The image displays two screenshots related to a FastAPI application. The top screenshot shows the FastAPI documentation interface for the 'default' API. It lists five endpoints: GET / Home, GET /tasks (Get Tasks), POST /tasks (Create Task), PUT /tasks/{index} (Update Task), and DELETE /tasks/{index} (Delete Task). The bottom screenshot shows a REST client interface with the URL 'http://127.0.0.1:8000/docs#/default/create_task_tasks_post'. It displays the curl command for a POST request to 'http://127.0.0.1:8000/tasks' with headers for 'accept' and 'Content-Type' set to 'application/json'. The request body is a JSON object: {'title': 'API Project', 'completed': true}. The server response is shown as a 200 status code with a response body: {'message': 'Task Added Successfully', 'task': {'title': 'API Project', 'completed': true}}. The response headers include 'content-length: 85' and 'content-type: application/json'.

Practical: 6**Aim – Add Custom Exception Handling.**

```
from fastapi import FastAPI, HTTPException
```

```
app = FastAPI()
```

```
students = {  
    1: "Nitish",  
    2: "Rahul",  
    3: "Amit"  
}
```

```
@app.get("/students/{id}")  
def get_student(id: int):
```

```
    if id not in students:
```

```
        raise HTTPException(  
            status_code=404,  
            detail="Student Not Found"  
        )
```

```
    return {  
        "Student Name": students[id]  
    }
```

FastAPI 0.1.0 OAS 3.1

/openapi.json

default

GET	/ Home
GET	/students Get Students

Execute Clear

Responses

Curl

```
curl -X 'GET' \n  'http://127.0.0.1:8000/students/5' \n  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/students/5
```

Server response

Code	Details
404	Error: Not Found

Response body

```
{  
  "detail": "Student Not Found"  
}
```

Download

Practical : 7**Aim – Add Background task to send async email**

```

from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import jwt, JWTError

```

```

app = FastAPI()

```

```

# =====
# JWT SETTINGS
# =====

```

```

SECRET_KEY = "mysecret"
ALGORITHM = "HS256"

```

```

# =====
# USERS
# =====

```

```

users = {

    "sachin": {
        "password": "1904",
        "role": "student"
    },

    "nitish": {
        "password": "2201",
        "role": "teacher"
    }
}

```

```

# =====
# OAUTH2 TOKEN
# =====

```

```

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="login"
)

```

```
# =====  
# LOGIN API  
# =====  
  
@app.post("/login")  
def login(  
    form_data: OAuth2PasswordRequestForm = Depends()  
):  
  
    username = form_data.username  
    password = form_data.password  
  
    # Check whether user exists  
  
    if username not in users:  
  
        raise HTTPException(  
            status_code=401,  
            detail="User not found"  
        )  
  
    # Check password  
  
    if users[username]["password"] != password:  
  
        raise HTTPException(  
            status_code=401,  
            detail="Wrong password"  
        )  
  
    # Get user role  
  
    role = users[username]["role"]  
  
    # Create JWT token  
  
    token_data = {  
        "username": username,  
        "role": role  
    }
```

```
token = jwt.encode(
    token_data,
    SECRET_KEY,
    algorithm=ALGORITHM
)

return {

    "access_token": token,
    "token_type": "bearer"

}

# =====
# GET CURRENT USER
# =====

def get_current_user(
    token: str = Depends(oauth2_scheme)
):

    try:

        # Decode JWT token

        data = jwt.decode(
            token,
            SECRET_KEY,
            algorithms=[ALGORITHM]
        )

        username = data.get("username")
        role = data.get("role")

        # Check token data

        if username is None or role is None:

            raise HTTPException(
                status_code=401,
                detail="Invalid token"
```

```
)

return {
    "username": username,
    "role": role
}

except JWTError:

    raise HTTPException(
        status_code=401,
        detail="Invalid token"
    )

# =====
# STUDENT API
# =====

@app.get("/student")
def student_api(
    user = Depends(get_current_user)
):

    # Check role

    if user["role"] != "student":

        raise HTTPException(
            status_code=403,
            detail="Only students can access this API"
        )

    return {

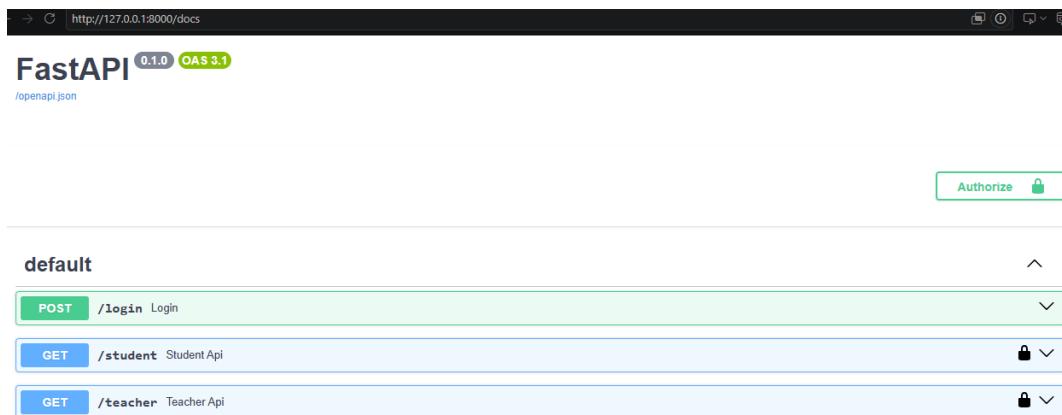
        "message": "Welcome Student",

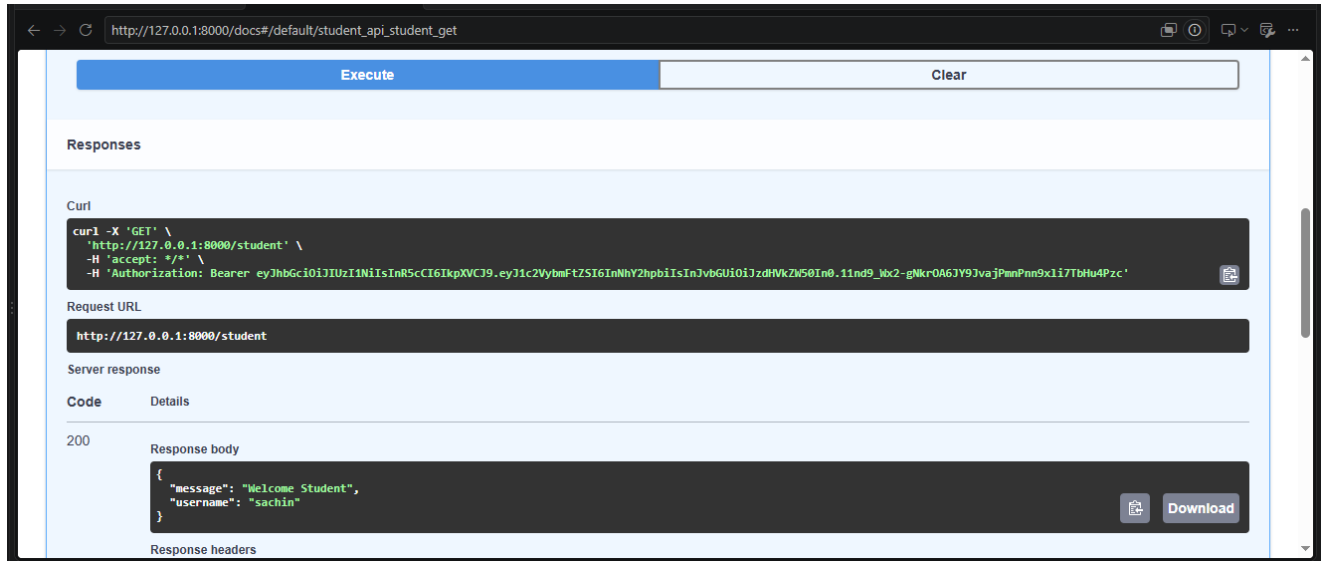
        "username": user["username"]

    }
```

```
# =====  
# TEACHER API  
# =====  
  
@app.get("/teacher")  
def teacher_api(  
    user = Depends(get_current_user)  
):  
  
    # Check role  
  
    if user["role"] != "teacher":  
  
        raise HTTPException(  
            status_code=403,  
            detail="Only teachers can access this API"  
        )  
  
    return {  
  
        "message": "Welcome Teacher",  
  
        "username": user["username"]  
    }  
}
```

Output:





Practical: 8**Aim – Secure routes with user roles**

```
# Import FastAPI
from fastapi import FastAPI

# Import BackgroundTasks
from fastapi import BackgroundTasks

# Used to simulate delay
import time

# Create FastAPI application
app = FastAPI()

# -----
# Background Task Function
# -----
def send_email(email: str):
    """
    This function simulates sending an email.
    It runs in the background.
    """

    print(f"✉ Sending email to: {email}")

    # Simulate email sending delay
    time.sleep(10)

    print("✓ Email sent successfully!")

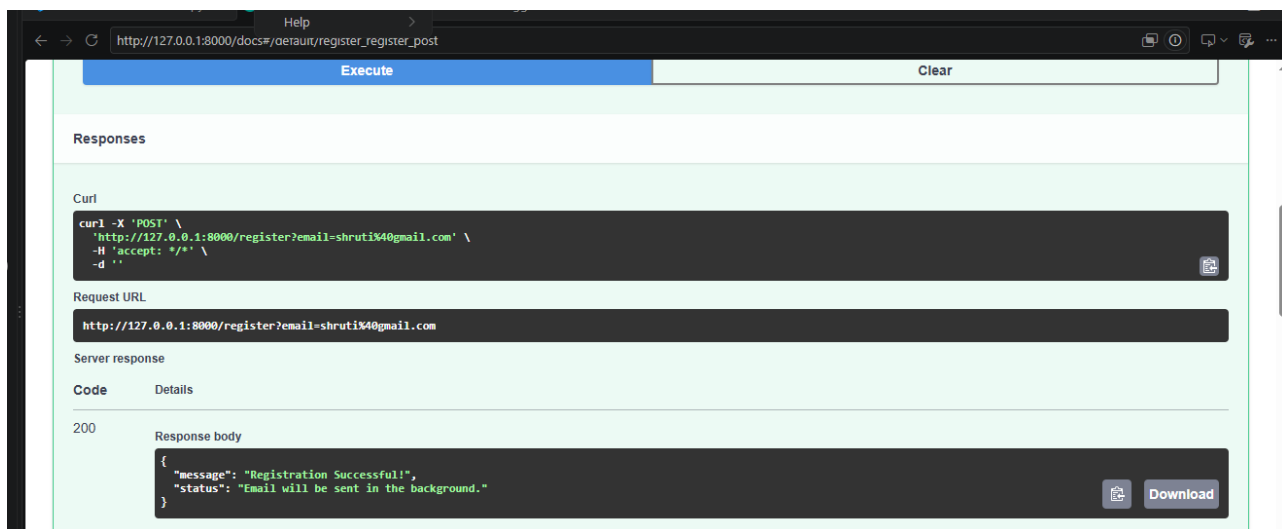
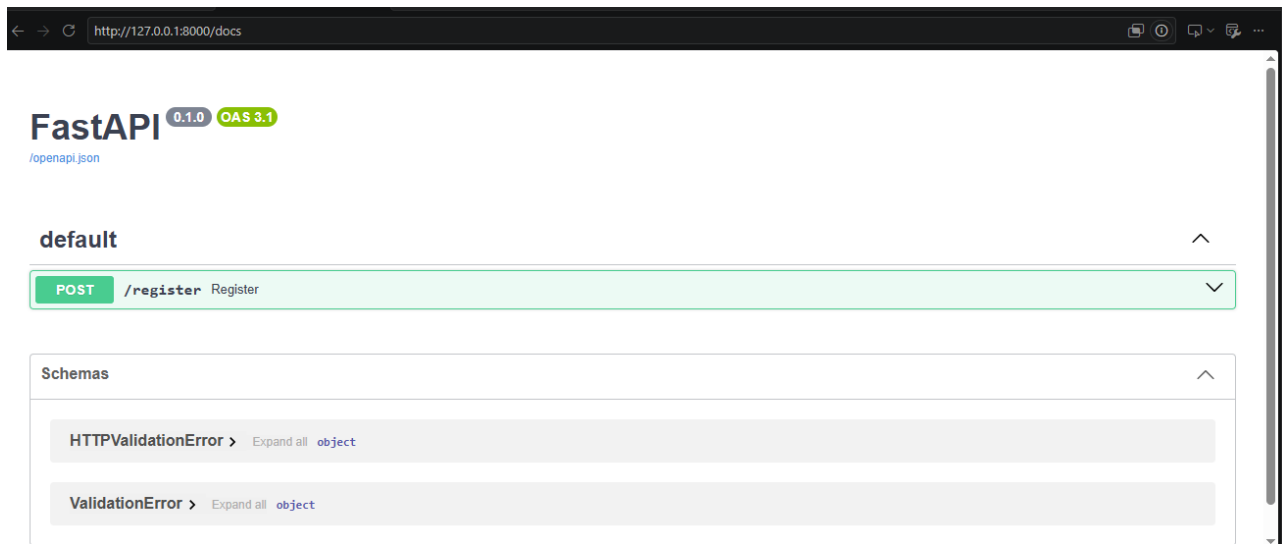
# -----
# API Endpoint
# -----
@app.post("/register")
async def register(email: str, background_tasks: BackgroundTasks):
    """
    Register a user and send a welcome email
    in the background.
    """

    print("👤 User registered successfully!")
```

```
# Add background task
background_tasks.add_task(send_email, email)

return {
    "message": "Registration Successful!",
    "status": "Email will be sent in the background."
}
```

Output:



Practical:9**Aim – Write integration test with Pytest/httpx**

```
from fastapi.testclient import TestClient
```

```
from main import app
```

```
client = TestClient(app)
```

```
def test_home():
```

```
    response = client.get("/")
```

```
    assert response.status_code == 200
```

```
    assert response.json() == {
        "message": "Welcome to Student API"
    }
```

```
def test_get_students():
```

```
    response = client.get("/students")
```

```
    assert response.status_code == 200
```

```
    assert len(response.json()) > 0
```

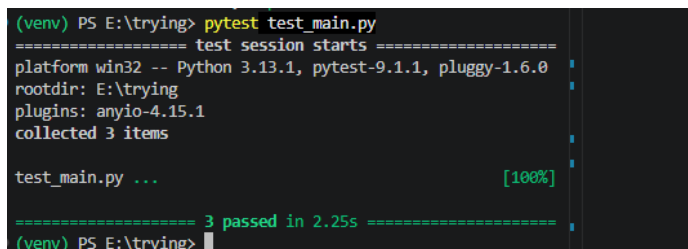
```
def test_get_existing_student():
```

```
    response = client.get("/students/1")
```

```
    assert response.status_code == 200
```

```
    assert response.json()["name"] == "Rahul"
```

Output:



```
(venv) PS E:\trying> pytest test_main.py
===== test session starts =====
platform win32 -- Python 3.13.1, pytest-9.1.1, pluggy-1.6.0
rootdir: E:\trying
plugins: anyio-4.15.1
collected 3 items

test_main.py ... [100%]

===== 3 passed in 2.25s =====
(venv) PS E:\trying>
```

Practical: 10

Aim- Create async DB APIs with Tortoise ORM

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from tortoise import fields
from tortoise.models import Model
from tortoise.contrib.fastapi import register_tortoise
```

```
app = FastAPI()
```

```
# Database Model
```

```
class Student(Model):
    id = fields.IntField(pk=True)
    name = fields.CharField(max_length=100)
    age = fields.IntField()
    email = fields.CharField(max_length=100)
```

```
class Meta:
    table = "students"
```

```
# Request Model
```

```
class StudentCreate(BaseModel):
    name: str
    age: int
    email: str
```

```
# CREATE
```

```
@app.post("/students")
async def create_student(student: StudentCreate):
```

```
    new_student = await Student.create(
        name=student.name,
        age=student.age,
        email=student.email
    )
```

```
    return {
        "message": "Student created successfully",
        "id": new_student.id,
        "name": new_student.name,
        "age": new_student.age,
```

```
        "email": new_student.email
    }
```

READ ALL

```
@app.get("/students")
async def get_students():

    students = await Student.all()

    return [
        {
            "id": student.id,
            "name": student.name,
            "age": student.age,
            "email": student.email
        }
        for student in students
    ]
```

READ ONE

```
@app.get("/students/{student_id}")
async def get_student(student_id: int):

    student = await Student.get_or_none(id=student_id)

    if student is None:
        raise HTTPException(
            status_code=404,
            detail="Student not found"
        )

    return {
        "id": student.id,
        "name": student.name,
        "age": student.age,
        "email": student.email
    }
```

UPDATE

```
@app.put("/students/{student_id}")
async def update_student(
    student_id: int,
```

```
student: StudentCreate
):

existing_student = await Student.get_or_none(
    id=student_id
)

if existing_student is None:
    raise HTTPException(
        status_code=404,
        detail="Student not found"
    )

existing_student.name = student.name
existing_student.age = student.age
existing_student.email = student.email

await existing_student.save()

return {
    "message": "Student updated successfully"
}
```

```
# DELETE
@app.delete("/students/{student_id}")
async def delete_student(student_id: int):

    student = await Student.get_or_none(
        id=student_id
    )

    if student is None:
        raise HTTPException(
            status_code=404,
            detail="Student not found"
        )

    await student.delete()

    return {
        "message": "Student deleted successfully"
    }
```

Tortoise Configuration

```
register_tortoise(  
    app,  
    db_url="sqlite://students.db",  
    modules={"models": ["main"]},  
    generate_schemas=True,  
    add_exception_handlers=True  
)
```

Output:

